

## 1. Core Combinatorics & Counting

These formulas are fundamental for calculating the number of possible combinations.

| Concept                | Formula                 | Explanation                                                                                |
|------------------------|-------------------------|--------------------------------------------------------------------------------------------|
| Subsets                | $2^n$                   | For each element, you have 2 choices (include it or not). With $n$ elements, it is $2^n$ . |
| Subsequences           | $2^n - 1$               | Same as subsets, but typically excludes the empty subsequence.                             |
| Substrings / Subarrays | $\frac{n * (n + 1)}{2}$ | The count of all contiguous segments.                                                      |
| Subarrays of size $k$  | $n - k + 1$             | A window of size $k$ can start at index 0 and slide up to index $n-k$ .                    |

## 2. Array Manipulation & Indexing

Essential tricks for handling array indices correctly.

| Technique                     | Formula                                                                                                                                                                                                                     | Use Case                                                                                                                                                                |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Circular Array                | <code>i % n</code>                                                                                                                                                                                                          | To wrap around an array. Often used in loops that go past the array length.                                                                                             |
| Negative Index Modulo         | <code>(i % n + n) % n</code>                                                                                                                                                                                                | Safely handles negative results from a modulo operation in languages like C++/Java.                                                                                     |
| Right Rotation                | <code>(i + k) % n</code>                                                                                                                                                                                                    | Finds the new index of element <code>i</code> after a right rotation by <code>k</code> positions.                                                                       |
| Left Rotation                 | <code>(i - k + n) % n</code>                                                                                                                                                                                                | Finds the new index of element <code>i</code> after a left rotation by <code>k</code> positions.                                                                        |
| Middle Index (0-indexed)      | <code>mid = n / 2</code>                                                                                                                                                                                                    | For an odd length array (e.g., $n=5$ , $mid=2$ ). This relies on integer division.                                                                                      |
| Middle Indices (0-indexed)    | <code>low = n / 2 - 1</code> , <code>high = n / 2</code>                                                                                                                                                                    | For an even length array (e.g., $n=6$ , $low=2$ , $high=3$ ). Lower Mid and Upper Mid                                                                                   |
| Converging Pointers           | <code>i = 0</code> , <code>j = n - 1</code><br>Access <code>arr[i]</code> and its symmetric partner <code>arr[j]</code> or <code>arr[n - 1 - i]</code> .                                                                    | Checking for palindromes, reversing an array in-place, Two Sum on a sorted array.                                                                                       |
| Fast & Slow Pointers          | <code>slow = arr[slow]</code> , <code>fast = arr[arr[fast]]</code>                                                                                                                                                          | Primarily for linked lists, but also for finding cycles in arrays where <code>arr[i]</code> points to another index. (e.g., LeetCode 287: "Find the Duplicate Number"). |
| Cyclic Sort                   | For arrays with numbers in a range (e.g., 1 to $n$ ).<br>Logic: <code>while (nums[i] != i + 1)</code> , swap <code>nums[i]</code> with the element at its correct position: <code>swap(nums[i], nums[nums[i] - 1])</code> . | Finding missing or duplicate numbers in a constrained range in $O(n)$ time. (e.g., "Find the Missing Number", "Find All Duplicates in an Array").                       |
| Dutch National Flag Partition | Three pointers: <code>low</code> , <code>mid</code> , <code>high</code> .<br>Partition an array into 3 sections ( <code>&lt; pivot</code> , <code>== pivot</code> , <code>&gt; pivot</code> ).                              | The core of the "Sort Colors" problem (LeetCode 75). Also the partitioning step in QuickSort.                                                                           |

### 3. Algorithmic Patterns

#### a. Subarray Contribution for Element `arr[i]`

Let's focus on an element at a generic index `i`.

Okay, let's visualize this with a diagram for `arr = [10, 20, 30, 40]` where `n = 4`.

Now, let's pick `arr[2]` (`30` in our example). So, `i = 2`.

```
Array:   [ 10, 20, 30, 40 ]
Index:    0   1   2   3   (n = 4)
          ^
          |
          i = 2 (Focus on element '30')
```

- 1. Possible Start Indices (from the Left)

These are all the indices `j` such that `j <= i`. The number of such indices is `i + 1`.

```
Array:   [ 10, 20, 30, 40 ]
Index:    0   1   2   3
          <-----^----->
          | Possible start points (j <= i)
          |
          Start at index 0
          Start at index 1
          Start at index 2 (This is 'i')
```

Number of possible start indices = `i + 1`  
= `2 + 1`  
= `3`

*Visual Representation of Possible Starting Points:* The `arr[i]` element (our `30`) can be part of a subarray that starts at:

- `arr[0]` (10)
- `arr[1]` (20)
- `arr[2]` (30 itself)

- 2. Possible End Indices (to the Right)

These are all the indices `k` such that `k >= i`. The number of such indices is `n - i`.

```
Array:   [ 10, 20, 30, 40 ]
Index:    0   1   2   3
          <-----^----->
                  | Possible end points (k >= i)
                  |
                  End at index 2 (This is 'i')
                  End at index 3
```

Number of possible end indices = `n - i`

$$= 4 - 2$$

$$= 2$$

*Visual Representation of Possible Ending Points:* The `arr[i]` element (our 30) can be part of a subarray that ends at:

- `arr[2]` (30 itself)
- `arr[3]` (40)

### • 3. Total Contribution of `arr[i]`

To form a subarray that *includes* `arr[i]`, we need to pick **one** of the possible start indices AND **one** of the possible end indices.

Since these choices are independent:

**Total subarrays containing `arr[i]` = (Number of Possible Start Indices) \* (Number of Possible End Indices)**

- For `arr[2]` (element 30):
  - $= (i + 1) * (n - i)$
  - $= (2 + 1) * (4 - 2)$
  - $= 3 * 2$
  - $= 6$

This confirms that the element 30 contributes to 6 subarrays.

**An O(n) technique to calculate sums over all subarrays without generating them.**

- **Logic:** Iterate through each element `arr[i]` and calculate how many subarrays it contributes to.
  - Number of possible start indices:  $i + 1$
  - Number of possible end indices:  $n - i$
- **Total Sum of All Subarrays:**
  - Contribution of `arr[i]`  $= arr[i] * (i + 1) * (n - i)$
  - Total Sum = The sum of these contributions for every element `i`.
- **Sum of All Odd-Length Subarrays:**
  - Contribution of `arr[i]`  $= arr[i] * (((i + 1) * (n - i) + 1) / 2)$  (using integer division).

## B. Binary Search

Used for searching in a sorted array or a monotonic search space.

- **Standard Middle (avoids overflow):**  $mid = low + (high - low) / 2$ 
  - Select the **lower-middle** element in an even-sized range.
- **Upper Middle:**  $mid = low + (high - low + 1) / 2$ 
  - Select the **upper-middle** element. Useful for avoiding infinite loops in certain search conditions.

## C. Sliding Window

Efficiently processes contiguous blocks of data.

- **Window Size:**  $window\_size = right\_pointer - left\_pointer + 1$
- **Frequency Maps/Arrays:** Use a hash map or an array to track element frequencies in the window.
  - Digits 0-9: `new int[10]`
  - Lowercase English letters: `new int[26]`
  - All ASCII characters: `new int[128]` or `new int[256]`

## D. Prefix Sum

Pre-calculates sums to answer range sum queries in O(1) time.

- `prefix[i] = arr[0] + ... + arr[i]`
- **Sum of Subarray [i, j] :** `prefix[j] - prefix[i-1]`
- **Problem: Find subarray sum equal to k**
  - Logic: `prefix[j] - prefix[i-1] = k`, which means `prefix[i-1] = prefix[j] - k`.
  - Use a hash map to check if the value `prefix[j] - k` has been seen before.
- **Problem: Find subarray sum divisible by k**
  - Logic: `(prefix[j] - prefix[i-1]) % k == 0`, which means `prefix[j] % k == prefix[i-1] % k`.
  - Use a hash map to count the frequencies of remainders (`prefix_sum % k`).

## Prefix Sum 2D

### 1. The Pre-computation Logic

In a 1D array, each element in the prefix array stores the sum from the start to that index. In 2D, `prefix[i][j]` stores the sum of all elements in the rectangle from `(0, 0)` to `(i, j)`.

To calculate this efficiently, we use the principle of inclusion-exclusion:

- **Add top neighbor:** `prefix[i-1][j]`
- **Add left neighbor:** `prefix[i][j-1]`
- **Subtract the diagonal overlap:** `prefix[i-1][j-1]` (because it was added twice by the neighbors).

### 2. Querying a Sub-rectangle Sum

To find the sum of a rectangle defined by top-left corner and bottom-right corner, we use a similar inclusion-exclusion logic:

- **Start with the total area:** `prefix[r2][c2]`
- **Subtract the area above:** `prefix[r1-1][c2]`
- **Subtract the area to the left:** `prefix[r2][c1-1]`
- **Add back the top-left corner:** `prefix[r1-1][c1-1]` (it was subtracted twice).

### 3. Example Walkthrough

#### Input Matrix

|   | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 1 | 4 | 5 | 6 |
| 2 | 7 | 8 | 9 |

#### Calculated Prefix Sum Matrix

|   | 0  | 1  | 2  |
|---|----|----|----|
| 0 | 1  | 3  | 6  |
| 1 | 5  | 12 | 21 |
| 2 | 12 | 27 | 45 |

Calculation Example for `prefix[1][1]` : `matrix[1][1]` (5) + `prefix[0][1]` (3) + `prefix[1][0]` (5) - `prefix[0][0]` (1) = 12.

#### Query Example

Find the sum of the sub-rectangle from **(1, 1)** to **(2, 2)** (Values: 5, 6, 8, 9).

- **Formula:** `prefix[2][2] - prefix[0][2] - prefix[2][0] + prefix[0][0]`

- **Substitution:**
- **Verification:**

## E. Greedy

### 1. Core Concept & Philosophy

| Concept                     | The "Formula" / Proof Strategy                                                                                                                                                                                                                                          | When to be Cautious                                                                                                                                                |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Greedy Choice</b>        | At each step, choose the best possible option <i>right now</i> without considering future consequences.                                                                                                                                                                 | If a local choice could negatively impact the range of future optimal choices, greedy will likely fail. This often indicates a <b>Dynamic Programming</b> problem. |
| <b>Proof of Correctness</b> | <p><b>1. Greedy Choice Property:</b> Prove that a locally optimal choice will be part of <i>some</i> globally optimal solution.</p> <p><b>2. Optimal Substructure:</b> Prove that an optimal solution to the problem contains optimal solutions to its subproblems.</p> | You don't need to formally prove this in an interview, but you must have a strong intuition for <i>why</i> your greedy choice works.                               |

### 2. The Sorting Heuristic (The Practical "Formula")

| Sort By...                                      | Greedy Choice & Rationale                                                                                                       | Common LeetCode Application                                                                                                                                             |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>End Time</b><br>(Ascending)                  | Choose the interval that <b>finishes earliest</b> . This frees up resources for other potential choices as quickly as possible. | <b>Activity Selection / Interval Scheduling.</b> Finding the maximum number of non-overlapping intervals. (e.g., "Non-overlapping Intervals").                          |
| <b>Start Time</b><br>(Ascending)                | Process items in a chronological or sequential order. This is for problems where the start order matters.                       | <b>Interval Merging.</b> (e.g., "Merge Intervals", "Meeting Rooms").                                                                                                    |
| <b>Value / Weight Ratio</b> (Descending)        | Choose the item with the most "bang for your buck". Maximizes value for a given capacity.                                       | <b>Fractional Knapsack.</b>                                                                                                                                             |
| <b>Value / Size</b> (Largest or Smallest First) | Handle the most constrained or most impactful items first.<br>e.g., Give the smallest cookie to the child with the least greed. | <b>Assignment Problems.</b> (e.g., "Assign Cookies").<br>e.g., Match the people with the lightest weights with the heaviest to fit in a boat. ("Boats to Save People"). |
| <b>A Custom Heuristic</b>                       | Sort by a problem-specific property. For example, <code>start_i + length_i</code> to find the farthest reachable point.         | <b>Jump Game II.</b> The greedy choice is to make the jump that offers the maximum reach for the <i>next</i> jump.                                                      |

## 4. Data Structures

### Array Representation Complete Binary Tree

Used in heaps and segment trees. Relies on integer division.

- **For 1-based indexing:** (Array index `1...n`)
  - Left child of node `i` : `2 * i`
  - Right child of node `i` : `2 * i + 1`
  - Parent of node `i` : `i / 2`
- **For 0-based indexing:** (Array index `0...n-1`)
  - Left child of node `i` : `2 * i + 1`

- o Right child of node  $i$  :  $2 * i + 2$
- o Parent of node  $i$  :  $(i - 1) / 2$

### General Binary Tree Properties

These formulas apply to any binary tree, where  $N$  is the total number of nodes.

| Property                                   | Formula                            | Description                                                                                         |
|--------------------------------------------|------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Total Edges</b>                         | $E = N - 1$                        | In a non-empty binary tree with $N$ nodes, there are always $N - 1$ edges (connections).            |
| <b>Max Nodes at Level <math>l</math></b>   | $2^l$                              | The maximum number of nodes possible at level $l$ (root is $l = 0$ ).                               |
| <b>Max Nodes for Height <math>h</math></b> | $N_{max} = 2^{h+1} - 1$            | The maximum total number of nodes in a binary tree of height $h$ (occurs in a perfect binary tree). |
| <b>Min Height for <math>N</math> Nodes</b> | $h_{min} = \lceil \log_2 N \rceil$ | The minimum possible height for a binary tree with $N$ nodes (occurs in a complete binary tree).    |

### Full Binary Tree Properties

A **Full Binary Tree** is a tree where every node has either 0 or 2 children.

| Property                         | Formula      | Description                                                                                                      |
|----------------------------------|--------------|------------------------------------------------------------------------------------------------------------------|
| <b>Leaves vs. Internal Nodes</b> | $L = T + 1$  | The number of leaf nodes ( $L$ ) is always one more than the number of internal nodes with two children ( $T$ ). |
| <b>Total Nodes</b>               | $N = 2T + 1$ | The total number of nodes ( $N$ ) in terms of internal nodes with two children ( $T$ ).                          |
| <b>Total Nodes</b>               | $N = 2L - 1$ | The total number of nodes ( $N$ ) in terms of leaf nodes ( $L$ ).                                                |

## Bit Manipulation

### XOR Properties

| Property                                    | Meaning                       |
|---------------------------------------------|-------------------------------|
| $a \oplus b = c \Rightarrow b \oplus c = a$ | You can reverse XOR           |
| $x \oplus 0 = x$                            | XOR with 0 returns same value |
| $x \oplus x = 0$                            | XOR with itself is 0          |

### Bit Shift Tricks

| Operation | Meaning       |
|-----------|---------------|
| $1 \ll n$ | Equals $2^n$  |
| $a \gg 1$ | Divide by 2   |
| $a \ll 1$ | Multiply by 2 |

### Set Operations Using Bitmask

The key idea is that **an integer can represent a set**. Each bit position (0, 1, 2, 3, etc.) corresponds to an element. If the bit is **1** ("on"), the element is **in** the set. If the bit is **0** ("off"), the element is **not** in the set.

Let's use two simple sets for all our examples:

- **Set A = 5**
  - Binary: `...00000101`
  - Bits 0 and 2 are `1`.
  - This represents the set `{0, 2}`.
- **Set B = 6**
  - Binary: `...00000110`
  - Bits 1 and 2 are `1`.
  - This represents the set `{1, 2}`.

Here is the refined table, explaining each operation using these two sets.

Simplified Set Operations (Bitmask)

| Operation                   | Code                                       | Simplified Explanation (Using A = {0, 2} and B = {1, 2} )                                                                                                                                                                                                                                                                                     |
|-----------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Union ( $A \cup B$ )        | <code>A   B</code>                         | <b>Goal:</b> Get all elements from A or B.<br><b>Set:</b> <code>{0, 2} ∪ {1, 2} = {0, 1, 2}</code><br><b>Bitmask:</b> <code>...0101   ...0110 = ...0111</code> (which is integer <code>7</code> )                                                                                                                                             |
| Intersection ( $A \cap B$ ) | <code>A &amp; B</code>                     | <b>Goal:</b> Get elements in <i>both</i> A and B.<br><b>Set:</b> <code>{0, 2} ∩ {1, 2} = {2}</code><br><b>Bitmask:</b> <code>...0101 &amp; ...0110 = ...0100</code> (which is integer <code>4</code> )                                                                                                                                        |
| Subtraction ( $A - B$ )     | <code>A &amp; ~B</code>                    | <b>Goal:</b> Get elements in A <i>but not</i> in B.<br><b>Set:</b> <code>{0, 2} - {1, 2} = {0}</code><br><b>Bitmask:</b> <code>...0101 &amp; ~...0110 = ...0101 &amp; ...1001 = ...0001</code> (integer <code>1</code> )                                                                                                                      |
| Negation<br>(Complement)    | <code>~A</code>                            | <b>Goal:</b> Get all elements <i>not</i> in A.<br><b>Set:</b> Complement of <code>{0, 2}</code> is <code>{1, 3, 4, 5, ...}</code><br><b>Bitmask:</b> <code>~...00000101 = ...11111010</code>                                                                                                                                                  |
| Set bit (Add)               | <code>A  = (1 &lt;&lt; n)</code>           | <b>Goal:</b> Add element <code>n</code> to the set.<br><b>Example:</b> Add element <code>3</code> to A ( <code>{0, 2}</code> ).<br><b>Bitmask:</b> <code>...0101   (1 &lt;&lt; 3) = ...0101   ...1000 = ...1101</code> (integer <code>13</code> ).<br><b>Result:</b> New set is <code>{0, 2, 3}</code> .                                      |
| Clear bit (Remove)          | <code>A &amp;= ~(1 &lt;&lt; n)</code>      | <b>Goal:</b> Remove element <code>n</code> from the set.<br><b>Example:</b> Remove element <code>2</code> from A ( <code>{0, 2}</code> ).<br><b>Bitmask:</b> <code>...0101 &amp; ~(1 &lt;&lt; 2) = ...0101 &amp; ~...0100 = ...0101 &amp; ...1011 = ...0001</code> (integer <code>1</code> ).<br><b>Result:</b> New set is <code>{0}</code> . |
| Test bit (Check)            | <code>(A &amp; (1 &lt;&lt; n)) != 0</code> | <b>Goal:</b> Check if element <code>n</code> is in the set.<br><b>Example:</b> Is element <code>2</code> in A ( <code>{0, 2}</code> )?<br><b>Bitmask:</b> <code>...0101 &amp; (1 &lt;&lt; 2) = ...0101 &amp; ...0100 = ...0100</code> (which is <code>4</code> ).<br><b>Result:</b> <code>4 != 0</code> , so <b>True</b> .                    |
| Extract last bit            | <code>A &amp; -A</code>                    | <b>Goal:</b> Get just the <i>smallest</i> element from the set.<br><b>Example:</b> Use B ( <code>{1, 2}</code> ), which is <code>...0110</code> .                                                                                                                                                                                             |

|                        |                |                                                                                                                                                                                                                                                                                                                   |
|------------------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                        |                | <b>Bitmask:</b> $\dots 0110 \& \sim \dots 0110 = \dots 0010$ (integer 2 ).<br><b>Result:</b> The set {1} . This isolates the smallest element, 1.                                                                                                                                                                 |
| Remove last bit        | $A \& (A - 1)$ | <b>Goal:</b> Remove the <i>smallest</i> element from the set.<br><b>Example:</b> Use B ( {1, 2} ), which is $\dots 0110$ .<br><b>Bitmask:</b> $\dots 0110 \& (\dots 0110 - 1) = \dots 0110 \& \dots 0101 = \dots 0100$ (integer 4 ).<br><b>Result:</b> The new set is {2} . The smallest element, 1, was removed. |
| All 1-bits (Universal) | $\sim 0$       | <b>Goal:</b> Get a set containing <i>all</i> possible elements.<br><b>Bitmask:</b> $\sim 0 = \dots 11111111$<br><b>Result:</b> The set {0, 1, 2, 3, 4, ...}                                                                                                                                                       |

Matrix / Grid Cheat Sheet

Directional Vectors

| Directions          | Use Case                  | Vectors (row, col)                                                             |
|---------------------|---------------------------|--------------------------------------------------------------------------------|
| 4 Directions (NSEW) | BFS, DFS, Flood Fill      | $\{-1, 0\}, \{1, 0\}, \{0, -1\}, \{0, 1\}$ (Up, Down, Left, Right)             |
| 8 Directions (All)  | Knight Moves, Word Search | $\{-1,-1\}, \{-1,0\}, \{-1,1\}, \{0,-1\}, \{0,1\}, \{1,-1\}, \{1,0\}, \{1,1\}$ |

Matrix ↔ Array Conversion

For an  $N \times M$  matrix (N rows, M columns).

| Operation     | Formula                             |
|---------------|-------------------------------------|
| 2D → 1D Index | $index = row * M + col$             |
| 1D Index → 2D | $row = index / M, col = index \% M$ |

Common Operations & Tricks

| Operation                    | Key Logic / Formula                                       |
|------------------------------|-----------------------------------------------------------|
| Transpose (In-place, Square) | <code>swap(matrix[i][j], matrix[j][i])</code> for $j > i$ |
| Rotate 90° Clockwise         | 1. Transpose matrix<br>2. Reverse each row                |
| Rotate 90° Anti-Clockwise    | 1. Transpose matrix<br>2. Reverse each column             |
| Sudoku 3x3 Box ID            | $box\_id = (row / 3) * 3 + (col / 3)$                     |